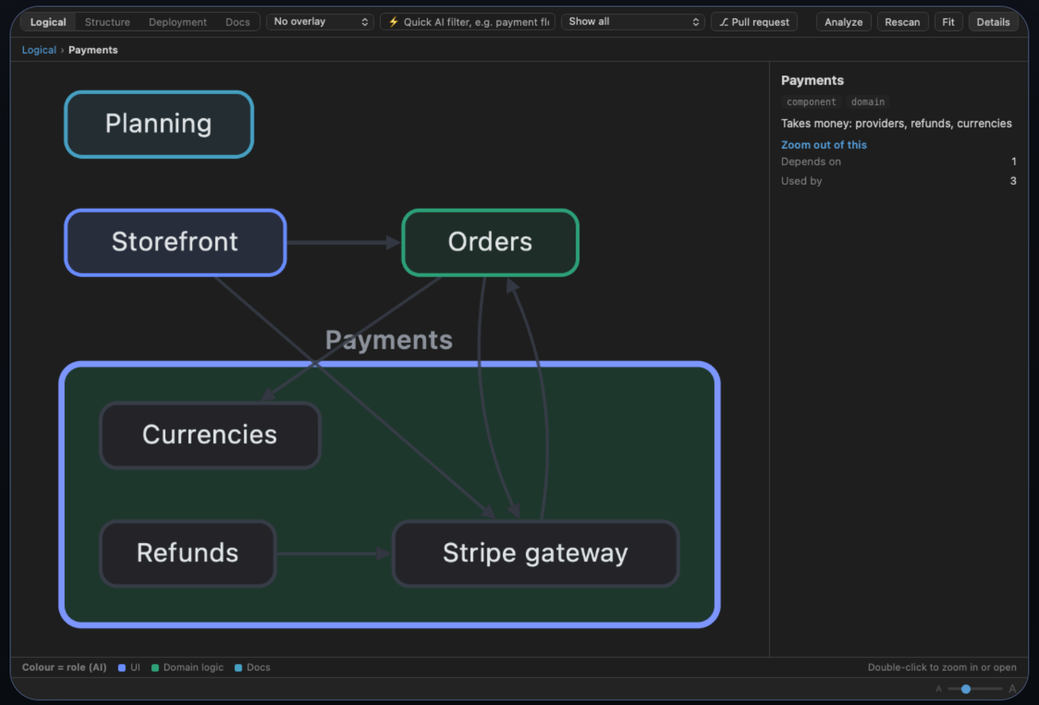
MarkView

Understand any project, then build it with AI.

A native macOS workspace for code and documentation: from idea to merged code.



- macOS 13+
- Signed & notarized
- MIT
- v5.11

THE PROBLEM

Projects outgrow their documentation, and AI agents work half blind.

Nobody sees the whole system

New people, and your own agents, open a repository and meet thousands of files with no map.

Specs live in chats and heads

Decisions get lost between a ticket, a conversation and a pull request. The code drifts away from the docs.

Agents need a workspace

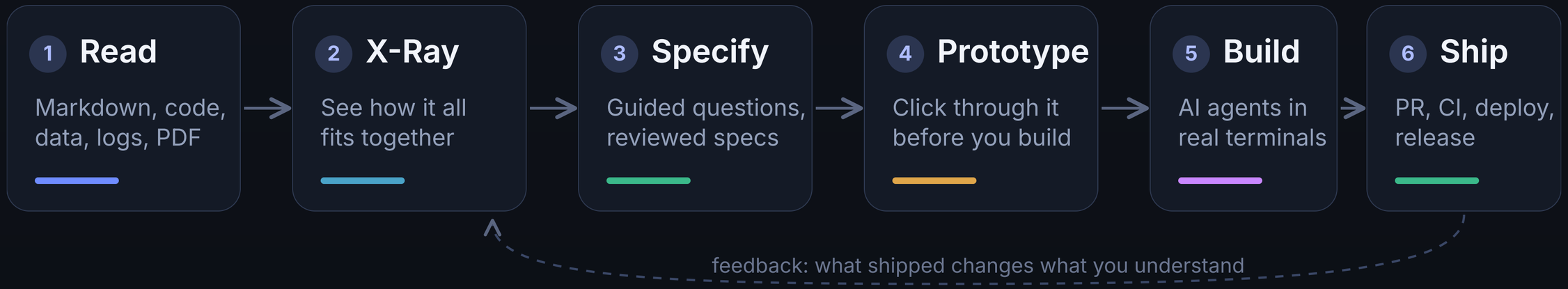
Claude Code, Codex and others are powerful, but a chat window is not where you understand, specify, review and ship.

MarkView puts the map, the specification, the prototype and the agent **in one window, on top of your own files**.

WHAT MARKVIEW IS

One workspace for the whole life of a change.

It starts as a plain Markdown reader and grows with you. Each step stands on its own: use only what you need.



Everything is a plain file in your repository (Markdown with YAML front matter). No server, no account, no lock-in.

STEP 1 · READ AND WRITE

A fast viewer for everything in a project.

Open one Markdown file or a whole folder. No project, AI or account required.

Markdown, WYSIWYG or source

Mermaid diagrams, KaTeX math, task lists, footnotes, front matter and `[[wikilinks]]`. Export to PDF, or build a whole PDF book.

Code in 40+ languages

CodeMirror 6 with syntax highlighting; press Edit to change it in place. Image, JSON Canvas, XML, plist and archive viewers.

Data as tables, with SQL

CSV, Parquet, Arrow, Avro, SQLite, Excel, HAR. Sort, filter, column statistics, and real SQL in the page.

JSON and YAML you can edit

Double-click a value in the tree. YAML keeps its comments, JSON keeps its indentation.

Logs that read themselves

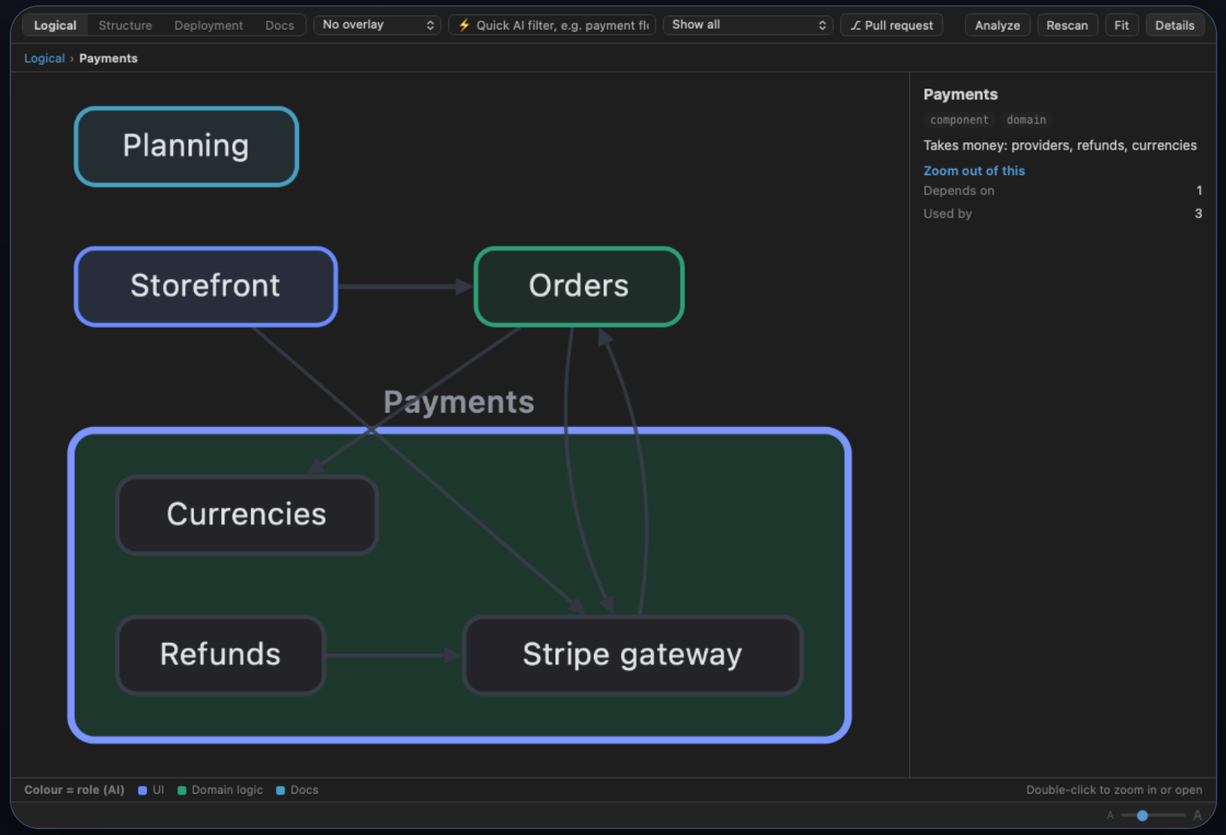
Levels with counts, stack traces, regex search, *Next error*, and *Follow* for a growing log.

Search the whole project

File names and file contents, plus commands, in one palette: ⌘ ⇧ K. Find in file with case and regex.

STEP 2 · UNDERSTAND

X-Ray: see how a project really fits together.



Logical view, zoomed into one component. Colour shows the role. (Earlier release: the toolbar has since gained the Code and Book views.)

- **Built from facts, named by AI.** The scan reads imports, links and git history, then clusters folders. AI only names and describes what the structure already shows.
- **Zoom all the way in.** Subsystem, component, file, section, a single function. Every leaf opens the file at that line.
- **Overlays** show documentation, tests, bug history, freshness, complexity, size, and what a pull request touches.
- **Five views:** Code, Logical, Structure, Deployment and Book.

STEP 2 · UNDERSTAND

Ask the project. Get evidence, not a guess.

Ask the project

Type a question in the ⚡ box. The answer comes as **What, Why, How and Origin**, with clickable evidence from code, documents, commits and pull requests. The relevant boxes light up.

Code architecture

Open a source folder and press `⌘` `4`: a layered map appears and sharpens in the background. Sequence diagrams trace a scenario, with clickable `path:line` evidence.

Book X-Ray and Explain

A folder of documents becomes parts, chapters and sections you can ask. Margin notes explain code and Markdown section by section.

PR X-Ray

See where a pull request lands in the architecture: a verdict, impact, findings you can fix with one click, chat, approve or merge. Works on a local branch too.

STEP 3 · SPECIFY

From a sentence to a specification that converges.

1 · Explore

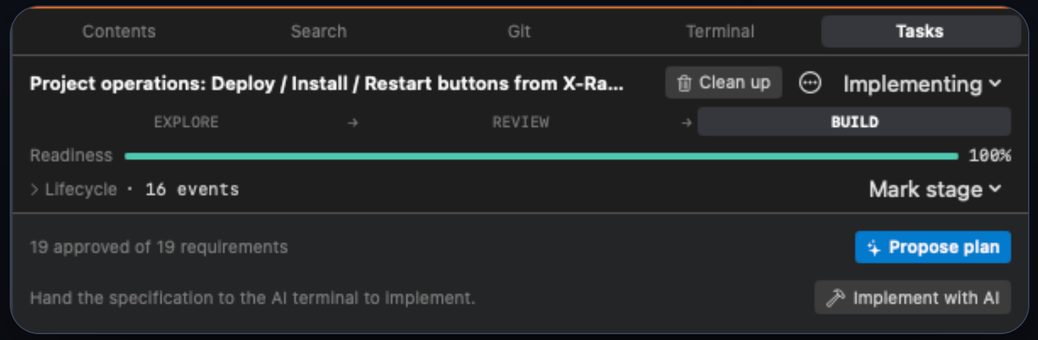
Rounds of at most three questions, each with concrete options and a recommended answer. The AI reads your code first, so it never asks what the project already answers.

2 · Review

The whole spec is read from a dozen perspectives. Every resolution is **written back into the requirements**, so the next review never raises a settled point again.

3 · Build

Propose a plan, create issues and an epic on GitHub, then *Implement with AI*: a binding handoff to the agent in the terminal.



Requirements, decisions, questions and findings are plain Markdown under `docs/features/<slug>/`: readable, diffable, reviewable in a pull request.

STEP 3 · CHECK

Reviews of every kind, one set of findings.

A specification

Twelve perspectives, from Product and UX to Security, QA and Privacy, with severity and quotes.

Docs against code

Finds statements the code no longer backs. The documentation overlay flags docs older than the code they describe.

Your changes

Bugs, edge cases, leftovers and missing tests with `file:line`. Observed file changes show agent edits side by side.

A pull request

Reviewed in its architecture context, with the diff, findings, impact and merge.

Security and the codebase

A security review prompt, plus Codebase Audit and Constructive Critic reports.

A prototype

Click through it, point at what to change, approve when it is right.

Review prompts never modify files: they report, and you decide.

STEP 4 · PROTOTYPE

Click through it before you build it.

- **Deep, not a picture.** Routes, forms with validation, roles, filters, empty and error states, and sample data that links across screens. Plain HTML, CSS and JavaScript.
- **Review in a studio tab.** A live preview next to a chat. Ask for a change in your own words, or by voice. Every accepted change is a new version.
- **Point at things.** Click *Point*, then an element: the assistant gets its HTML and, if you like, a screenshot. Paste a sketch or another product.
- **Approve and hand off.** A ZIP with the prototype, `SPEC.md`, `CHANGES.md` and a README: an implementer, human or AI, builds from it.

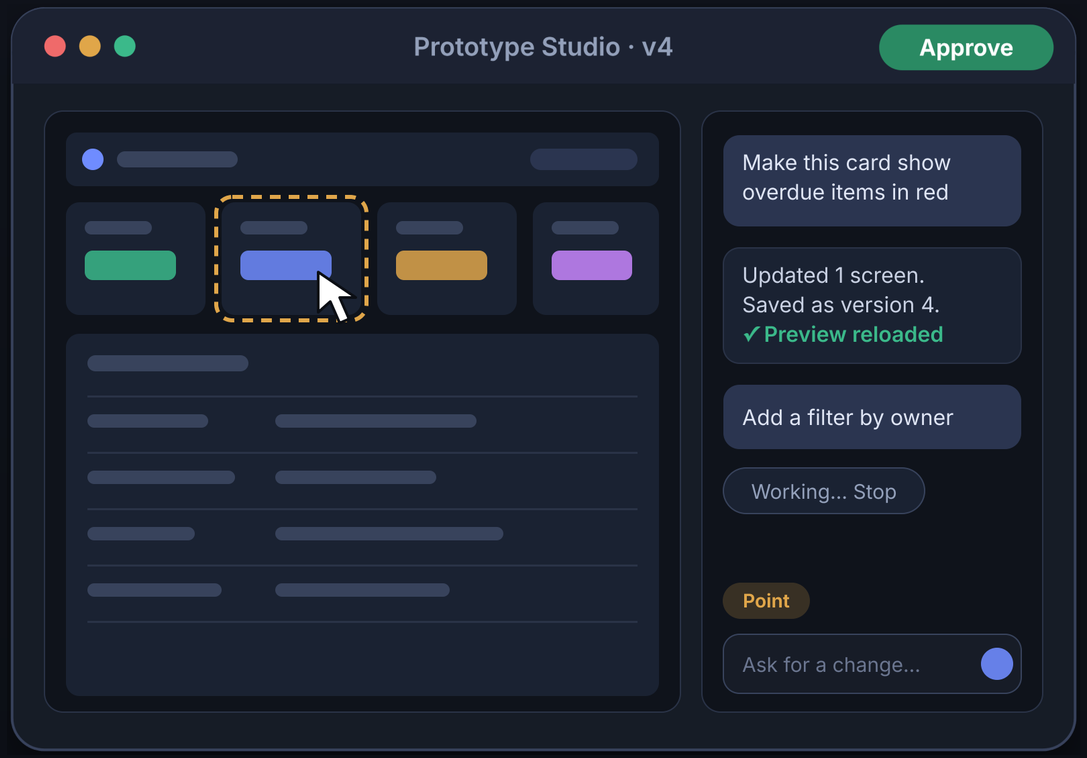
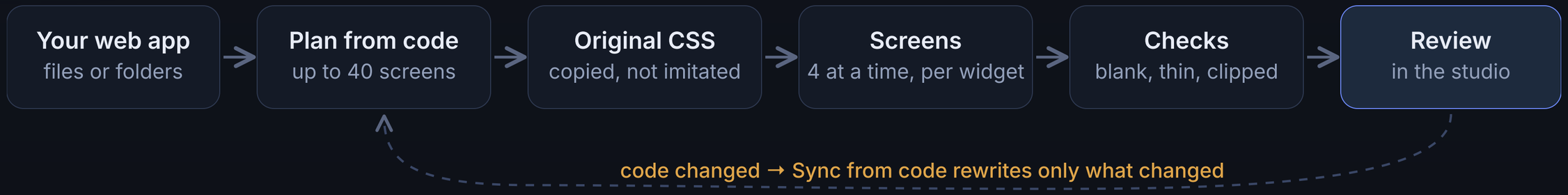


Illustration of the studio tab: live preview, chat, versions.

STEP 4 · PROTOTYPE

Start from your code, and stay in step.



Faithful look

The app's own stylesheets, fonts and images are copied as they are. A theme picker switches the app's themes.

Honest limits

A *From code* badge, plus *Assumptions & gaps* that say what was not reproduced.

Sync status

A chip tells whether the prototype *matches the code*. *Sync from code* keeps the previous version.

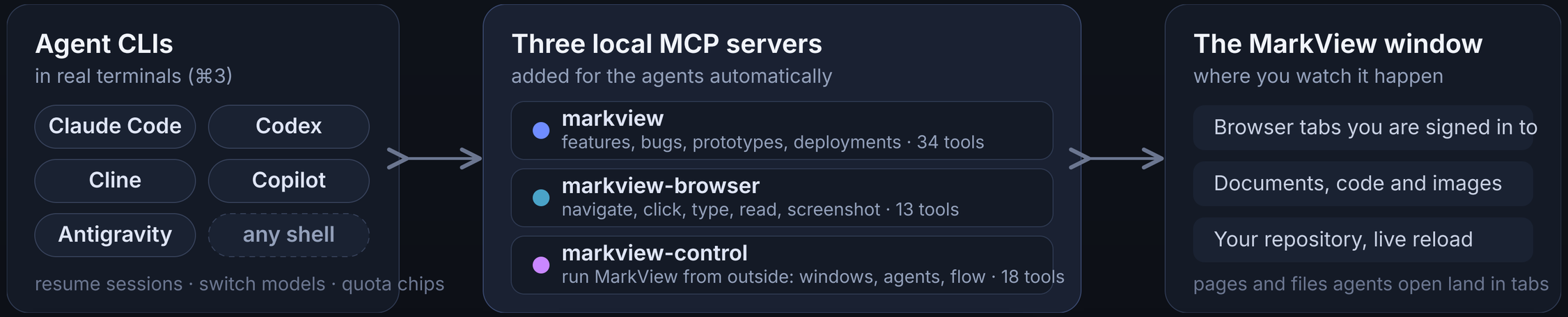
Hand off to code

An editable brief goes to an agent that makes a branch and a pull request. The studio never writes to your source.

STEP 5 · BUILD

Your agents, inside your workspace.

Claude Code, Codex, Cline, GitHub Copilot and Antigravity run in real terminals next to your files. They use your own sign-in: MarkView needs no provider key.



They drive your browser tabs

Sign in once, name the tab "Jira", say *"in tab Jira, follow these steps"*. No password is ever shared. **Stop** takes the tab back.

They understand MarkView

"Turn this prototype into features." The app writes the files in its own exact format, so ids and links stay valid.

One click, one prompt

Review changes, Docs ⇔ code, Find bugs, Write tests, Security review, Commit message, and more.

STEP 6 · SHIP AND RUN

See where it runs, and act on it safely.

Deployments (`^` `⌘` `D`) shows each place the project runs as one picture.

Finds where it runs

Reads workflows, platform files, deploy scripts, docs and `~/.ssh/config`: servers, Firebase, Vercel, Fly.io, Kubernetes, AWS, Google Cloud, and the databases behind them.

The whole picture

CPU, memory, disks, services and containers, your own green and red *checks*, logs, and one health badge. Ask the AI: *"Why is it slow?"*

Look into the database

Tables, sizes, indexes and a SQL box on PostgreSQL. Reads run in a read-only transaction. The password never appears in a command line.

Safe by construction

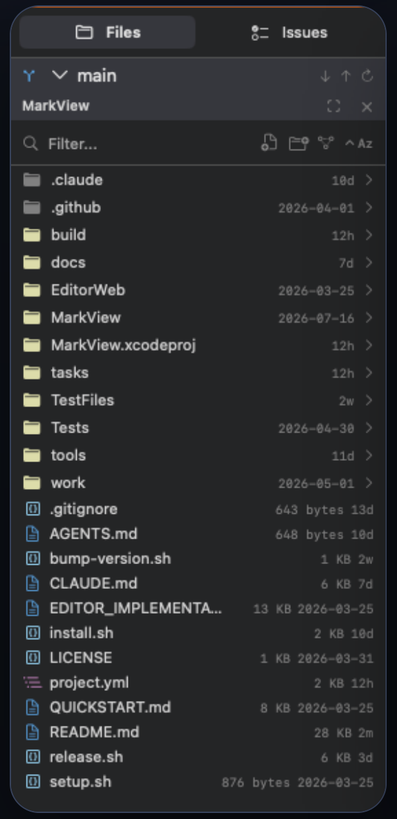
Read-only
uptime, df, `docker ps`, logs: run at once

Anything else
a restart, an edit, a deploy: you approve that exact command

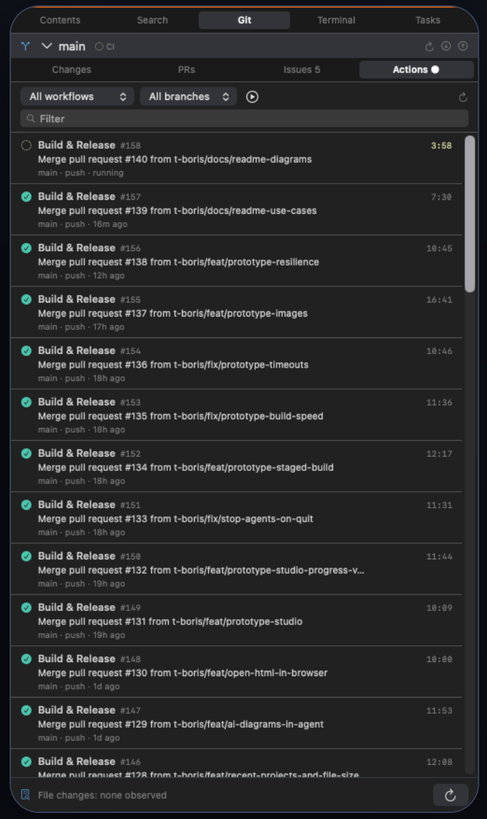
Never
shut a machine down, wipe a disk, pipe a download into a shell

STEP 6 · SHIP

Git, GitHub and a definition of done, built in.



File tree



Git panel: Actions

- **One window, one project.** Tabs, panels and size come back after a restart. Project colours tell windows apart, even in Mission Control.
- **Git in the app:** branches, stage, commit, push, pull, diffs, worktrees, and AI help with conflicts.
- **GitHub, opt-in:** pull requests with review and merge, issues, and Actions runs with *Explain Failure* and *Fix with AI*.
- **A definition of done** for *Implement with AI*: 12 items, each *Require*, *Ask me* or *Skip*. The agent must show evidence for every required one.
- **Project Cycle Time** shows where time goes, from idea to merge.

UNDER THE HOOD

Native, local, and yours to inspect.

236

Swift files, about 85,000 lines. SwiftUI and AppKit with a WKWebView editor.

0

Swift package dependencies. The web libraries ship inside the app.

5

agent CLIs and 3 local MCP servers: markview, markview-browser, markview-control.

96

standalone check scripts for the pure modules: lifecycle, prototypes, deployments, PDF books and more.

- **Files are the database.** Features, bugs, research and decisions are Markdown with YAML front matter in your repository. Machine state lives in a regenerable `.dde/` folder.
- **AI through the CLIs you already use.** No provider key, no hosted backend. MarkView's own background runs are read-only.
- **Opt-in integrations.** GitHub is off until you turn it on; when it is off, MarkView is silent about it.
- **Signed and notarized** with a Developer ID, so it opens without warnings. MIT licence.

SEE IT LIVE

A two-minute tour.

1
Open a folder
⌘ ⇧ O A file tree, git status and project search in ⌘ ⇧ K.

2
X-Ray it
⌘ 4 Layers fill in. Click a scenario for a sequence diagram with source links.

3
Ask the project
Type in the ⚡ box: *"How is a document saved?"* Get a cited answer and the boxes light up.

4
Query a data file
Open a CSV or Parquet file, click **SQL**, run a query with ⌘ ↵.

5
Run an agent
⌘ 3 Start Claude Code, then press *Review changes*.

6
Prototype from code
AI Tools → Prototype → *New Prototype from Code…* Then watch the build.

Bonus: ^ ⌘ D for Deployments, ⌘ 6 to preview a web app, ⌘ ⇧ E to export a PDF book.

GET MARKVIEW

Understand it. Specify it. Build it.

Download the signed, notarized installer, drag it to Applications, and open any folder.

⬇ MarkView.dmg

`github.com/t-boris/MarkView/releases/latest`

Requirements

- macOS 13 Ventura or later
- For AI: one or more agent CLIs, installed and signed in (Claude Code, Codex, Cline, GitHub Copilot or Antigravity)
- Optional: `git`, and `gh` for GitHub

Version 5.11 · MIT licence
Open source on GitHub